

NPR Shader Technique Analysis

Cross-Avatar Technical Comparison — Jade · Meta Avatar · Avaturn

Thesis Project · Quest 3 · Unity URP

2026

Overview

This report documents the technical implementation of eight Non-Photorealistic Rendering (NPR) techniques across three avatar pipelines used in the thesis study. Each technique is examined for execution space, kernel algorithm, buffer access, and per-pipeline differences that affect the visual output.

Avatar Pipelines

Pipeline	Scene	Shader Model	Edge Detection Runs
Jade (ShaderExperimental)	Kuwahara and hierarchical.unity	One standalone .shader per technique	Inside the vertex-lit fragment shader
Meta Avatar	Avaturn.unity (via Meta SDK)	Single monolithic shader, multi_compile keywords	Inside Meta SDK AppSpecificPostManipulation hook (end of PBR pass)
Avaturn	Avaturn.unity	One standalone .shader per technique	Inside the vertex-lit fragment shader

Both Jade and Avaturn also benefit from two shared **screen-space post-process** renderer features registered in `URP_QUEST_Renderer.asset` (the project's active renderer):

- `KuwaharaFilterFeature` → `AnisotropicKuwahara.shader` (4-pass, full screen)
- `EdgeDetectionFeature` → `HierarchicalEdgeDetection.shader` (depth + normals textures)

Pipeline Architecture Comparison

Aspect	Jade	Meta Avatar	Avaturn
Has real depth buffer	No	No	No (but post-process does)
Has normals texture	No	No	Yes — DepthNormals pass writes to <code>_CameraNormalsTexture</code>
Inverted-hull outline	Pass 0 of each shader	Separate <code>OUTLINE_PASS</code> define	Pass 0 of each shader
Toon shading	Optional toggle in V1, V4	Applied to full PBR output	Not in standalone shaders
Normal map support	V2, V4 (Fresnel + Sobel normal edge)	None (vertex interpolation only)	All shaders (TBN in ForwardLit + DepthNormals)
Post-process benefits	Kuwahara + Hierarchical (full screen)	Kuwahara + Hierarchical (full screen)	Kuwahara + Hierarchical (full screen, + normal-map normals)
Active URP renderer	<code>URP_QUEST_Renderer.asset</code>	<code>URP_QUEST_Renderer.asset</code>	<code>URP_QUEST_Renderer.asset</code>

Technique-by-Technique Analysis

1 — Outline Hull with Toon Shading

Implementation: Inverted-hull geometry expansion for the silhouette outline. The main pass applies step-quantized diffuse shading (posterisation).

	Jade V1	Meta EFFECT_TOON	Avaturn
Outline mechanism	Normal-expanded vertex pass (<code>posWS + normalWS × width</code>)	Same inverted hull via <code>OUTLINE_PASS</code> define	Same inverted hull
Toon algorithm	<code>floor(lum × steps + 0.5) / steps</code> on raw Lambert	<code>floor(rgb × bands + 0.5) / bands</code> on full PBR output	Not implemented
Rim light	<code>1 - NdotV</code> rim added on top	None	None
Key parameters	<code>_ToonSteps</code> (1–10), <code>_ToonThreshold</code> , <code>_OuterOutlineWidth</code>	<code>_ToonColorBands</code> (2–8), <code>_ToonPosterizeStrength</code>	—

Key difference: Meta applies posterisation to the full PBR result (GI, specular, shadows), so cel bands look richer but are less controllable. Jade applies toon to raw Lambert only.

Gap: Avaturn has no toon standalone shader.

2 — Normal Edge Detection (+ optional Fresnel)

Implementation: Detects geometric discontinuities using screen-space derivatives of world-space normals. Fresnel silhouette is the `1 - NdotV` band at grazing angles.

	Jade V2	Meta EF- EFFECT_NORMAL_EDGE	Avaturn
Normal source	<code>ddx/ddy(normalSample)</code> — samples the normal map	<code>ddx/ddy(worldNormal)</code> — vertex interpolation only	Not implemented
Fresnel	<code>1 - NdotV</code> double-smoothstep band	Same double-smoothstep band	—
Normal map sensitivity	Yes — fires on normal-map ridges (eyelid crease, lip fold)	No — vertex normals only, all detail smoothed out	—
Key parameters	<code>_NormalEdgeThreshold</code> , <code>_FresnelEdgeThreshold</code> , <code>_FresnelEdgeStrength</code>	<code>_NormalEdgeThreshold</code> , <code>_FresnelEdgeThreshold</code> , <code>_FresnelEdgeStrength</code>	—

Key difference: Jade's normal edge detects edges baked in the normal map — view-independent surface detail. Meta's fires only on geometric silhouettes.

Note on Fresnel: Both pipelines expose `_FresnelEdgeStrength = 0` to disable it. It is

sensitive to camera zoom because it fires at silhouette angles — making it optionable is recommended for documentation.

Gap: Avaturn has no normal edge shader (relies on post-process `EdgeDetectionFeature`).

3 — Sobel Edge Detection

Implementation: 3×3 Sobel operator on base colour texture luminance. Computes horizontal and vertical gradients, takes magnitude.

$$\text{Sobel}_X = (tr + 2r + br) - (tl + 2l + bl), \quad \text{Sobel}_Y = (tl + 2t + tr) - (bl + 2b + br)$$

$$\text{edge} = \sqrt{\text{Sobel}_X^2 + \text{Sobel}_Y^2}$$

	Jade V3	Meta EFFECT_SOBEL	Avaturn V3
Kernel	Standard 3×3 Sobel	3×3 Sobel + seam spike suppression	Standard 3×3 Sobel
Source texture	<code>_MainTex</code> (albedo luminance)	<code>u_BaseColorSampler</code> (Meta UV atlas)	<code>_MainTex</code> (albedo luminance)
Seam suppression	None	<code>step(lMax - lMin, _SeamLimit)</code>	None
Normal map	No	No	No (but <code>DepthNormals</code> pass exports normal-map normals to post-process)
Key parameters	<code>_InnerLineThreshold</code> , <code>_InnerLineBlur</code> , <code>_InnerLineStrength</code>	<code>_SobelThreshold</code> , <code>_SobelMax</code> , <code>_SobelSeamLimit</code> , <code>_SobelStrength</code>	<code>_EdgeThreshold</code> , <code>_EdgeSampleDist</code> , <code>_InnerLineStrength</code>

Key difference: Meta adds seam suppression — critical because the Meta Avatar UV atlas has hard seams where the Sobel gradient spikes, producing false outlines. Jade and Avaturn don't need it.

4 — Gaussian Sobel

Implementation: Sobel is preceded by an optional 9-tap Gaussian pre-blur at each of the 9 kernel positions. A configurable threshold band, 4 progressive smoothstep passes, and a power curve are applied after the gradient magnitude.

	Jade V4	Meta EF- FECT_GAUSS_SOBEL	Avaturn
Pre-blur	9-tap Gaussian at each Sobel position (configurable weights)	Identical	Not implemented as standalone

	Jade V4	Meta EF- FECT_GAUSS_SOBEI	Avaturn
Extra edge layers	+ ddx/ddy(worldNormal) + Fresnel silhouette	None — purely texture-based	—
Threshold band	smoothstep(min, max, edgeMag)	Same + 4 progressive passes + power curve	—
Key parameters	_BlurRadiusMultiplier, GSobelEnableGaussBlur, _GaussianCenterWeight, GSobelBlurRadius, _GaussianCardinalWeight, GSobelThreshold, _GaussianDiagonalWeight, GSobelTightness, _ThresholdMinMultiplier, GSobelPowerCurve, _ThresholdMaxMultiplier, GSobelStrength 4× _EnablePassN		

Key difference: Jade V4 adds world-space normal and Fresnel edge layers on top of Gaussian Sobel; Meta’s version is purely texture-based.

Gap: Avaturn has no Gaussian Sobel standalone shader — relies on the shared `EdgeDetectionFeature` (hierarchical, not pure Gaussian Sobel).

5 — Halftone / Hatching

Implementation:

- **Halftone** — Rotating dot grid; tone-responsive radius $r = \sqrt{\max(0, 1 - \text{tone})} \times 0.5$; smooth circle edge via `smoothstep`.
- **Hatching** — 4-layer Tonal Art Map (Praun et al.): primary angle ($t > 0.15$), cross angle ($t > 0.35$), dense diagonal ($t > 0.55$), fill ($t > 0.80$). Each layer activated progressively by darkness.

	Jade (HalftoneHatching)	Meta EF- FECT_HALFTONE / HATCHING	Avaturn VHH
Halftone algorithm	Rotating dot grid; sqrt radius formula	Identical	Identical (same shader file)
Hatching algorithm	4-layer TAM	Identical	Identical (same shader file)
Tone source	NdotL × shadow + _ToneBias (Lambert)	Full PBR output luminance	Same as Jade (Lambert)
UV space	Object UV, or world-space XZ/XY/YZ blend	Object UV only	Object UV
Outline	Yes (separate pass)	Yes (via OUTLINE_PASS)	Yes
Normal map	DepthNormals pass present	No	DepthNormals pass present

	Jade (HalftoneHatching)	Meta EF- FECT_HALFTONE / HATCHING	Avaturn VHH
Key parameters	<code>_HalftoneScale,</code> <code>_HatchScale,</code> <code>_HatchThickness,</code> <code>_ToneLevels,</code> <code>_ToneBias</code>	<code>_HTScale,</code> <code>_HTSharpness,</code> <code>_HatScale,</code> <code>_HatThickness,</code> <code>_HatToneBias</code>	Same as Jade

Key difference: Meta derives tone from the full PBR pipeline (physically lit, including GI and shadows), so dots/lines respond to area lights. Jade/Avaturn use simpler Lambert tone. The Avaturn VHH uses the **identical shader file** as Jade — no implementation difference.

6 — Kuwahara

Implementation: Anisotropic Kuwahara filter. Structure tensor (Sobel on luminance) is computed, smoothed with Gaussian blur, then eigenanalysis gives orientation φ and anisotropy A . An elliptical neighbourhood is divided into 8 sectors; each sector's inverse-variance-weighted mean is the output colour.

	Jade (post-process)	Meta EF- FECT_KUWAHARA	Avaturn (post-process)
Implementation location	<code>KuwaharaFilterFeature</code> — 4-pass pipeline in screen space	<code>MPREffect_Kuwahara2</code> — inside fragment shader	Same as Jade (shared <code>URP_QUEST_Renderer</code>)
Structure tensor	Sobel on full scene luminance in screen space	<code>ddx/ddy(luminance)</code> per fragment — no kernel	Same as Jade
Tensor Gaussian blur	5-tap separable Gaussian (Pass 1)	None	Same as Jade
Sector sampling	8 sectors $\times$ 3 radii = 24 taps, elliptical	8 sectors $\times$ 3 radii = 24 taps	Same as Jade
Scope	Full screen (<code>avatarLayer = 0</code>)	Avatar mesh only	Full screen
Key parameters	<code>_KernelSize</code> (2–16), <code>_SectorCount,</code> <code>_Sharpness,</code> <code>_Hardness,</code> <code>_ZeroCrossing</code>	<code>_K2Radius,</code> <code>_K2Strength,</code> <code>_K2Alpha,</code> <code>_K2Q,</code> <code>_K2Tau</code>	Same as Jade

Key difference: The screen-space version (Jade/Avaturn post-process) produces true oil-paint abstraction with a properly smoothed structure tensor, resulting in stable oriented strokes. The Meta in-shader version has a noisy tensor (no pre-smoothing) — it produces a weaker, less coherent effect.

7 — Hierarchical Edge Detection

Implementation: Multi-layer edge detector combining depth, normal, and colour cues, fused by weighted max-pooling. An adaptive suppression factor based on local brightness reduces edges in highlights.

$$\text{edge} = \max(\text{depth} \times w_d, \max(\text{normal} \times w_n, \text{colour} \times w_c)) \times \text{adaptive}$$

	Jade / Avaturn (post-process)	Meta EFFECT_HIERARCHICAL
Depth layer	Roberts Cross on _CameraDepthTexture — true linear depth	length(worldViewDir) + ddx/ddy — approximate camera-distance proxy
Normal layer	Roberts Cross on _CameraNormalsTexture	ddx/ddy(worldNormal) — vertex normals only
Colour layer	Roberts Cross on scene colour buffer (_BlitTexture)	Roberts Cross on u_BaseColorSampler (UV texture lookup)
Gaussian preblur	Not on screen buffers	Optional 9-tap Gaussian on colour samples before Roberts Cross
Avatar-background edge	Yes — depth texture sees the avatar-background boundary	No — no depth buffer access, cannot detect silhouette from background
Normal map support	Yes (reads _CameraNormalsTexture; Avaturn V3/V5 writes normal-map normals there)	No — vertex normals only
Skin discard	No	Yes — HSV hue gate suppresses colour edges on skin pixels
Adaptive suppression	lerp(1, saturate(brightness × 2), strength)	Identical formula
Key parameters	depthThreshold, normalThreshold, colorThreshold, depthWeight, normalWeight, colorWeight, edgeWidth, adaptiveStrength	_HDepthThreshold, _HNormalThreshold, _HColorThreshold, _HDepthWeight, _HNormalWeight, _HColorWeight, _HEdgeWidth, _HAdaptiveStrength

Key difference: Post-process version detects the avatar silhouette against the background (depth texture). Meta’s in-shader version only sees edges on the avatar surface. The Avaturn V3 and V5 shaders write normal-map-perturbed normals to _CameraNormalsTexture, so the post-process edge pass sees normal-map detail on Avaturn but not on Jade.

8 — Hierarchical Edge Detection with Gaussian Pre-blur

Implementation: Same three-layer hierarchical pipeline as Technique 7, with the addition of a 9-tap Gaussian kernel applied to each of the 4 Roberts Cross colour samples before computing the gradient. This smooths high-frequency texture noise in the colour layer while preserving depth and normal discontinuities.

New shaders created for Jade and Avaturn:

	Jade V5_HierarchicalGauss	Avaturn V5_HierarchicalGauss	Meta EF- FECT_KUW_GAUSS_HIER (phased)
Depth layer	ddx/ddy(length(posWS - cameraPosWS))	Identical	ddx/ddy(length(worldViewDir))
Normal layer	ddx/ddy(worldNormal)	ddx/ddy(TBN-transformed worldNormal) (normal map aware)	ddx/ddy(worldNormal)
Colour layer	Roberts Cross on _MainTex luminance, each tap optionally pre-blurred	Identical	Roberts Cross on u_BaseColorSampler, optional 9-tap Gaussian
Gaussian toggle	_EnableGaussBlur	_EnableGaussBlur	_K2HEnableGaussBlur
Gaussian radius	_HBlurRadius	_HBlurRadius	_K2HBlurRadius
Outline pass	Yes	Yes	Yes
DepthNormals pass	No	Yes	No
Gaussian weights	_HCenterWeight, _HCardinalWeight, _HDiagonalWeight	Same	_K2HCenterWeight, _K2HCardinalWeight, _K2HDiagonalWeight

Turning off _EnableGaussBlur degenerates to pure Hierarchical (Technique 7 in-shader equivalent), enabling direct A/B comparison in the Inspector.

Gap Analysis Summary

Technique	Jade	Meta Avatar	Avaturn
Outline + Toon	Yes V1	Yes EFFECT_TOON	No Missing
Normal Edge (+ Fresnel)	Yes V2 (normal-map aware)	Yes (vertex only)	No Missing
Sobel	Yes V3	Yes (seam-suppressed)	Yes V3
Gaussian Sobel	Yes V4	Yes EF- FECT_GAUSS_SOBEL	No Missing (post-process only)
Halftone / Hatching	Yes HalftoneHatching	Yes EF- FECT_HALFTONE / HATCHING	Yes VHH (same shader as Jade)
Kuwahara	Yes Post-process (full screen)	Yes In-shader (weaker, no tensor blur)	Yes Post-process (same as Jade)
Hierarchical	Yes Post-process (depth texture)	Yes In-shader (approx. depth)	Yes Post-process + normal map from V3
Hierarchical + Gaussian	Yes V5 (new, depth+normal+Gauss Roberts Cross)	Yes EF- FECT_KUW_GAUSS_HIER phase 2	Yes V5 (new, + normal map + DepthNormals pass)

Research Finding: Face Component Extraction for NPR Abstraction

Research only — not implemented.

NPR effects applied uniformly to the face produce the same stroke density on forehead, nose, and lips. For artistically expressive results, eyes, mouth, and facial contours benefit from heavier or differentiated treatment.

Approach 1 — UV Semantic Masks (*most practical*)

Avatar UV atlases place facial regions (eyes, brows, lips, nose) in consistent UV coordinate bands. The Meta Avatar SDK documents this layout. A 1-channel semantic mask texture sampled at the same UV can modulate edge strength or Kuwahara radius per region with zero additional render cost beyond one texture lookup.

Approach 2 — Normal Direction Filtering

The nose tip, lip ridge, and brow ridge have normals pointing toward the camera while cheeks point laterally. A filter `dot(worldNormal, viewDir) > threshold` isolates the face centre from profile regions. Zero-cost and purely geometric, but unstable under head rotation.

Approach 3 — Depth Layer Isolation

In a head-on view the face occupies a distinct depth band closer than the ears and neck. A soft depth gate in the post-process edge pass (similar to the existing `_FadeWithDistance` parameter in `HierarchicalEdgeDetection.shader`) can amplify edges only within face depth — no shader changes required.

Approach 4 — Inverted HSV Skin-Hue Gate (*already partially implemented*)

The skin discard in `NPREffect_Hierarchical.cginc` (`_HEnableSkinDiscard`) suppresses colour edges on skin pixels. The gate can be **inverted** — running a stronger edge pass specifically on skin-hue pixels to amplify surface detail on the face while leaving clothing edges unchanged.

Approach 5 — Face Mesh Landmarks (ML)

MediaPipe Face Mesh outputs 468 facial landmarks in real time. Running this via Unity Barracuda/Sentis would give per-landmark semantic access (eye corners, lip outline, nose tip). Too heavy for Quest 3 real-time, but would enable the most precise per-region stylisation. Out of scope for the current study.

Recommendation for Thesis Documentation

Approach 1 (UV semantic mask) combined with **Approach 4** (inverted HSV skin gate) are the most actionable. Both work within the existing Meta shader hook without adding render passes, and they create a measurable perceptual difference in how each NPR technique reads on the face versus the body.

File Reference

File	Pipeline	Purpose
Assets/AvatarShaderExperimental/Shader/Shader/V1_ToonShadingToonShading.shader	Assets/AvatarShaderExperimental/Shader/Shader/V1_ToonShadingToonShading.shader	Geometric shading; no post-process
Assets/AvatarShaderExperimental/Shader/Shader/V2_NormalEdgeDetection.shader	Assets/AvatarShaderExperimental/Shader/Shader/V2_NormalEdgeDetection.shader	Normal map Sobel + normal ddx/ddy + Fresnel
Assets/AvatarShaderExperimental/Shader/Shader/V3_SobelEdgeDetection.shader	Assets/AvatarShaderExperimental/Shader/Shader/V3_SobelEdgeDetection.shader	Depth 3x3 Sobel
Assets/AvatarShaderExperimental/Shader/Shader/V4_GaussianPreFilteredSobel.shader	Assets/AvatarShaderExperimental/Shader/Shader/V4_GaussianPreFilteredSobel.shader	Filtered Sobel; Sobel + normal + Fresnel
Assets/AvatarShaderExperimental/Shader/Shader/V5_HierarchicalGaussian.shader	Assets/AvatarShaderExperimental/Shader/Shader/V5_HierarchicalGaussian.shader	Hierarchical (depth+normal+Gauss Roberts Cross)
Assets/AvatarShaderExperimental/Shader/Shader/4-pass post-process Kuwahara.shader	Assets/AvatarShaderExperimental/Shader/Shader/4-pass post-process Kuwahara.shader	4-pass post-process Kuwahara
Project/AnisotropicKuwahara.shader	Project/AnisotropicKuwahara.shader	
Assets/AvatarShaderExperimental/Shader/Shader/Post-process hierarchical edge detection.shader	Assets/AvatarShaderExperimental/Shader/Shader/Post-process hierarchical edge detection.shader	Post-process hierarchical edge detection
Project/HierarchicalEdgeDetection.shader	Project/HierarchicalEdgeDetection.shader	
Assets/AvatarShaderExperimental/Shader/Shader/Halftone dot grid + TAM hatching.shader	Assets/AvatarShaderExperimental/Shader/Shader/Halftone dot grid + TAM hatching.shader	Halftone dot grid + TAM hatching
Project/HalftoneHatching.shader	Project/HalftoneHatching.shader	
Assets/Shader/NPR/V3_SobelEdgeDetection.shader	Assets/Shader/NPR/V3_SobelEdgeDetection.shader	Sobel + DepthNormals pass
Assets/Shader/NPR/V5_HierarchicalGaussian.shader	Assets/Shader/NPR/V5_HierarchicalGaussian.shader	Hierarchical + Gaussian + DepthNormals pass
Assets/Shader/CustomShaders/AvatarMeta-UGB.shader	Assets/Shader/CustomShaders/AvatarMeta-UGB.shader	Monolithic shader; all techniques via multi_compile
Assets/Shader/CustomShaders/NPR/Affect_*.cginc	Assets/Shader/CustomShaders/NPR/Affect_*.cginc	Per-technique implementation files (13 total)
Assets/AvatarShaderExperimental/Script/Rendering/Kuwahara.shader	Assets/AvatarShaderExperimental/Script/Rendering/Kuwahara.shader	URP Render Features: screen-space Kuwahara
Assets/AvatarShaderExperimental/Script/Rendering/EdgeDetection.shader	Assets/AvatarShaderExperimental/Script/Rendering/EdgeDetection.shader	URP Render Features: screen-space hierarchical edges
Assets/URP_QUEST_Renderer.asset	Assets	Active URP renderer; contains both post-process features
Assets/URP_QUEST.asset	All scenes	Active URP pipeline; depth + opaque textures enabled